HOCHSCHULE MITTWEIDA
University of Applied Sciences

BACHELORARBEIT

Herr Paul Fuchs

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

2026

Fakultät Angewandte Computer- und Biowissenschaften

BACHELORARBEIT

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

Autor:
Paul Fuchs

Seminargruppe:
MI20w2-B

Studiengang:
Medieninformatik und
Interaktives Entertainment

Erstprüfer:
Prof. Philipp Klimant

Zweitprüfer:
Dr. Beier-Grunwald

Einreichung:
Mittweida, den 27.03.2026

Bibliografische Angaben

Fuchs, Paul: Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen. 41 Seiten, 13 Abbildungen, 1 Anlage mit Quelltexten, Hochschule Mittweida (FH), Fachbereich Angewandte Computer- und Biowissenschaften

Bachelorarbeit 2026

Abstract

In the current world of software engineering developers are measured by the frequency of software releases while operations have to maintain a stable and reliable system thus creating a discrepancy in time-to-market and reliability. While DevOps has the methodology to create collaboration between these team, Site Reliability Engineering implements these principles in practice. With the introduction of an error budget team now can balance between frequent releases and reliable systems. Simply put by - Do we still have budget? Yes, then go release. No, then we maintain the reliability until we have some budget available again. This thesis analyses how the consumption of error budget can be reduced by using canary releases instead of rolling updates and further analyse the canary update with white-box monitoring.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Forschungsfrage 2

2 Theoretische Grundlagen **3**

2.1 Microservice Architektur 3

2.2 Microservice Orchestrierung mit Kubernetes 4

2.3 Deployment Strategien 5

2.3.1 Rolling Updates 5

2.3.2 Canary 5

2.4 Site Reliability Engineering (SRE) 5

2.4.1 Konzepte und Prinzipien 6

2.4.2 Monitoring und Observability 7

2.5 Downtime im Sinne dieser Arbeit 8

3 Konzeption und Methodik **9**

3.1 Systemarchitektur 9

3.2 Theoretisches Modell 11

3.3 Parameter zur Analyse 11

3.3.1 Fehlerfreier Zustand des Systems 12

3.3.2 Fehlerszenario A 12

3.3.3 Fehlerszenario B 12

3.3.4 Analyseintervalle 12

4 Evaluation **15**

4.1 Analyse der injizierten Fehlerrate 15

4.1.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit 15

4.1.2 Die einzelnen Testläufe im zeitlichen Verlauf 17

4.1.3 Die gemittelten Werte der Deployment Strategien im Vergleich 19

4.2 Analyse der injizierten Latenz 19

4.2.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit 19

4.2.2 Die einzelnen Testläufe im zeitlichen Verlauf 20

4.2.3 Die gemittelten Werte der Deployment Strategien im Vergleich 22

4.3 Fehlerbudget Verbrauch pro Deployment Strategie 23

4.3.1 Fehlerszenario A 23

4.3.2 Fehlerszenario B 23

5 Diskussion **25**

5.1 Der erste Testlauf der injizierten Fehlerrate für das Canary Release mit globaler Metrikanalyse 25

5.2 Nicht gemessene Fehler während der ersten Canary Phase 25

5.3 Detektionszeit in Zusammenhang des Fehlerbudgetverbrauchs . . 26

6 Fazit **27**

6.1 Beantwortung der Forschungsfrage 27

6.2 Ausblick 27

A Eigenständigkeitserklärung **31**

Abbildungsverzeichnis

3.1 Systemarchitektur: EKS Cluster Versuchsaufbau 10

4.1 Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Fehlerrate 16

4.2 Fehlerrate Rolling Update im Einzelvergleich 17

4.3 Fehlerrate Canary Release globaler Metrikanalyse im Einzelvergleich 18

4.4 Fehlerrate Canary Release isolierter Analysemetrik im Einzelvergleich 18

4.5 Fehlerrate unterschiedlicher Deployment Strategien 19

4.6 Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Latenz 20

4.7 Latenz Rolling Update im Einzelvergleich 21

4.8 Latenz Canary Release globaler Metrikanalyse im Einzelvergleich 21

4.9 Latenz Canary Release isolierter Analysemetrik im Einzelvergleich 22

4.10 Latenzen unterschiedlicher Deployment Strategien 22

4.11 Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Fehlerrate über die Testdauer 23

4.12 Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Latenz über die Testdauer 24

Kapitel 1

Einleitung

In der modernen Welt der Softwareentwicklung werden Entwicklerteams daran gemessen, wie schnell sie neue Features in das bestehende System implementieren können, während operative Teams daran gemessen werden, wie stabil und zuverlässig das System läuft, welches sie operieren. Hier entsteht eine Diskrepanz zwischen Entwicklern, die ihre entwickelten Softwareanpassungen möglichst auf die Produktivumgebung ausrollen möchten und den Operativen Teams, die sicherstellen müssen, dass das System weiterhin zuverlässig funktioniert. Während DevOps Methoden die Theorie für die Kollaboration der beiden Teams beschreiben, ist Site Reliability Engineering die angewandte Praxis dieser Methoden. Mit der Einführung eines Fehlerbudgets wird eine Brücke zwischen schnellen Veröffentlichungen und stabilem System gebaut. Das Fehlerbudget ermöglicht es beiden Teams ein Gleichgewicht in der Erfüllung ihrer Ziele zu finden. Ist das Fehlerbudget ausgereizt, werden keine Veröffentlichungen ausgerollt, ist jedoch eine ausreichende Menge an Fehlerbudget übrig, kann das Risiko des Ausrollens einer neuen Softwareversion eingegangen werden. In dieser Arbeit wird analysiert, wie der Verbrauch des Fehlerbudgets bei der Bereitstellung neuer Softwareversionen mithilfe von Analysen weiter reduziert werden kann, damit selbst nach einer fehlerbehafteten Releases ausreichend Fehlerbudget für weitere Softwareupdates zur Verfügung steht.

1.1 Forschungsfrage

In welchem Maß verringert ein Canary Release mit Service-Level-Objective basierten Analysen den Verbrauch des Fehlerbudgets im Vergleich zu Rolling Updates mit statischen Alerts? Diese Arbeit soll quantitativ analysieren, welcher Anteil des Fehlerbudgets bei der Wahl von Deployment Strategie und zugrunde liegender analysierter Metrik eingespart werden kann.

Kapitel 2

Theoretische Grundlagen

Den Titel in Zusammenhang mit der Forschungsfrage lesend, stellen sich zunächst einige Fragen. Was sind Microservices und wodurch zeichnen sie sich aus? Möglicherweise, Was ist Kubernetes? Was ist die Bereitstellung eben jener Microservices? Weiter stellen sich die Fragen, Was sind Service-Level-Objectives, Was ist das Fehlerbudget, Was ist Downtime und zuletzt, Wie kann das alles gemessen werden. Die folgenden Abschnitte beantworten diese Fragen und führen zum Versuchsaufbau hin.

2.1 Microservice Architektur

James Lewis und Martin Fowler beschrieben bereits 2014 Microservice Architekturen und grenzten dabei den Architekturansatz klar von monolithischen Anwendungen ab. In aller Kürze beschreiben die Microservice Architektur als Ansatz bei dem eine Anwendung aus dem Verbund mehrerer Services besteht, von denen jeder als eigener Prozess läuft und deren Kommunikation über leichtgewichtige Mechanismen, bspw. HTTP-Ressource API's stattfindet. Dabei werden die Services auf Geschäftsprozesse zugeschnitten und lassen sich unabhängig voneinander testen, bereitstellen und skalieren. Weiter beschreiben sie für Microservice Architekturen die folgenden Charakteristika:

- Komponenten durch Services
- Organisierung anhand der Geschäftsprozesse
- Produkte, keine Projekte

- Schlaue Endpunkte und Nachrichtendepots
- Dezentrale Steuerung
- Dezentrales Datenmanagement
- Automatisierung der Infrastruktur
- Fehlertoleranz
- Evolutionäres Design

Diese Charakteristika lassen sich Ihrer Aussage nach bei den meisten Microservice Architekturen finden, sind jedoch nicht allgemeingültig oder als Definition zu Verstehen. (Vgl. [7])

2.2 Microservice Orchestrierung mit Kubernetes

In der Dokumentation zu Kubernetes ist nachzulesen, dass Kubernetes eine Open-Source Softwarelösung ist, die containerisierte Applikationen und Services verwaltet. In Produktionsumgebungen sollen die Container, die zur Lauffähigkeit des Systems beitragen, verwaltet werden und es müsse sichergestellt sein, dass es keine Downtime gibt. Dabei beschreiben die Autoren, dass Kubernetes ein Framework sei, welches verteilte Systeme resilient laufen lasse. Es kümmere sich darum, dass die Skalierung und Failover Mechanismen der Applikation bereitgestellt werde. Einige der Hauptaufgaben, die Kubernetes übernehmen könne sind, Service Erreichbarkeit und Lastverteilung, automatisierte Veröffentlichungen und Rollbacks und Selbstheilung. (Vgl. [9]) Die kleinsten ausrollbaren Einheiten in Kubernetes sind Pods. Diese dienen zu Ausführung der containerisierten Applikation und können von übergeordneten Komponenten verwaltet werden. (Vgl. [11]) Eine dieser übergeordneten Komponenten von Pods sind Services. Sie dienen als nach außen gerichtete Endpunkte und verteilen den ankommenden Datenverkehr auf die ihm zugeordneten Pods. (Vgl. [10])

2.3 Deployment Strategien

Die Autoren der Argo-Rollouts Dokumentation listen die gängigsten Deployment Strategien in ihren Konzepten auf, dazu gehören Rolling, Recreate, Blue-Green und Canary.(Vgl. [1]) Im folgenden werden die für diese Arbeit wichtigen Strategien erklärt.

2.3.1 Rolling Updates

Die Kubernetes Autoren beschreiben die Standard-Strategie eines Deployment Objektes sinngemäß so, dass die Pods einer nach dem anderen ausgetauscht werden. (Vgl. [8]) Verständlicher formuliert, wird ein Pod mit neuer Softwareversion lauffähig gemacht und nachdem dieser lauffähig ist, wird einer der Pods mit der alten Softwareversion heruntergefahren. Dieser Vorgang wird so lange wiederholt, bis alle Pods auf die neue Softwareversion aktualisiert wurden.

2.3.2 Canary

Die Argo-Rollouts Dokumentationsautoren beschreiben ein Canary Deployment so, dass nur ein Bruchteil der Nutzer Zugriff auf die neue Softwareversion erhält und dieser Nutzeranteil kontinuierlich ansteigt, bis festgestellt wurde, dass die Softwareversion korrekt läuft. Danach findet die flächendeckende Akutalisierung aller Services auf die neue Softwareversion statt, zu der alle Nutzer Zugang erhalten. (Vgl. [2]) Danilo Sato erklärt in den Fußnoten seiner Beschreibung für Canary Releases die Herkunft des Namens dieser Strategie. Er beschreibt, dass Minenarbeiter einen Kanarienvogel als Warnsignal in die Kohleminen mitgenommen haben. (Vgl. [5])

2.4 Site Reliability Engineering (SRE)

Die AWS und RedHat Autoren beschreiben Site Reliability Engineering als Praxis, die mithilfe von Software Lösungen infrastrukturelle Prozesse automatisiert. Diese beinhalten unter anderem Systemverwaltung, Applikationsmonitoring und andere operative Aufgaben. Weiter wird beschrieben, dass diese Praxis dabei hilft, dass

skalierbare Softwaresysteme, aufgrund der Automatisierung manueller Prozesse, zuverlässiger werden. (Vgl. [3, 13]) Die Redhat Autoren beschreiben weiter, dass SRE ein Gleichgewicht zwischen hochfrequenter Veröffentlichung neuer Features und der Zuverlässigkeit für Nutzer schafft.

2.4.1 Konzepte und Prinzipien

Chris Jones et al. beschreiben in der Einführung des Site Reliability Engineering Buchs für Kapitel 2 ”Principles”die Kernkonzepte für SRE. Dazu gehört Risiken einzugehen, die Definition von Service Level Objectives (SLO’s), die Automatisierung infrastruktureller Prozesse, Monitoring der Applikationen und die Entwicklung einer Veröffentlichungsstrategie. (Vgl. [6]) Folgend werden die Punkte Risiken eingehen, Defintion von SLO’s und Monitoring der Applikationen genauer beschrieben.

Risiken eingehen

Zuerst beschreiben Alvidrez und Guliani, dass von einem Unternehmen wie Google erwartet würde, dass die Services zu 100% zuverlässig wären. Doch im Folgenden wird festgestellt, dass ab einem bestimmten Punkt die Erhöhung der Zuverlässigkeit einer Software zur Verschlechterung des Services und dessen Nutzern führt. Damit gehe einher, dass neue Features weniger häufig veröffentlicht werden könnten, während Nutzer den Unterschied von hoher zu extremer Verlässlichkeit kaum spürten. (Vgl. [12]) Im letzten Abschnitt beschreiben Roth und Quinito die Motivation für ein Fehlerbudget. Dabei erklären sie, dass ein Fehlerbudget als objektive Metrik genutzt wird, um einzuschätzen, wie unzuverlässig ein System während einer bestimmten Zeitperiode sein darf. Der Hauptvorteil eines solchen Fehlerbudgets wird als Gleichgewicht von Innovation und Zuverlässigkeit beschrieben. So lange sich das System innerhalb des Fehlerbudgets bewegt, können neue Features veröffentlicht werden, wird das Budget jedoch überschritten, finden innerhalb des definierten Zeitraums keine weiteren Veröffentlichungen statt. (Vgl. [12])

Service Level Objectives

Zuerst werden von Jones et al. Terminologien definiert. Die Service Level Indicators werden als definiertes quantitatives Maß eines Aspekts des Services beschrieben, bspw. Latenz oder Fehlerrate. Service Level Objectives werden als Zielwert oder Wertebereich von Service Level Indicators, bspw. die Fehlerrate ist weniger als ein Prozent, definiert. Service Level Agreements werden als Vertragsverbindlichkeiten beschrieben, deren Nicht-Einhaltung meist finanzielle Konsequenzen nach sich trägt. (Vgl. [4]) Für diese Arbeit sind die Service Level Indicators (SLI) und Service Level Objectives (SLO's) von Bedeutung, da sie den Rahmen erlaubter bzw. kalkulierter Downtime eingrenzen.

2.4.2 Monitoring und Observability

Ewaschuk und Beyer beschreiben das Monitoring verteilter Systeme als SSammeln, Verarbeiten, Zusammenführen und Anzeigen quantitativer Echtzeitdaten"[14] eines Systems. Weiter wird zwischen White-Box Monitoring, dem Auslesen Service-interner Metriken und Black-Box Monitoring, dem außerhalb sichtbaren Verhalten, "wie ein Nutzer es sehen würde"[14] unterschieden. Folgend werden die vier goldene Signale des Monitoring beschrieben, dazu gehören:

- Latenz
- Datenverkehr
- Fehler
- Sättigung

(Vgl. [14])

Die AWS Autoren beschreiben Observability für SRE als Prozess der Vorbereitung auf Unsicherheiten, wenn eine Software Nutzern zur Verfügung gestellt wird. Dabei werden Metriken, Logs und Traces gesammelt und ausgewertet, um Schwachstellen zu identifizieren und zu beheben. (Vgl. [3])

2.5 Downtime im Sinne dieser Arbeit

In Wikipedia wird Downtime als ”die gebräuchliche Bezeichnung der Zeit, in der ein System, insb. ein Computersystem, nicht verfügbar bzw. nicht funktionstüchtig ist. Man unterscheidet zwischen geplanter und ungeplanter Downtime.”[15] definiert. Die geplante Downtime in dieser Arbeit wird der Zeit zugeordnet, die sich innerhalb der definierten SLO’s bewegt. Jede Nicht-Verfügbarkeit des Systems über diese Zeit hinaus wird der ungeplanten Downtime zugeordnet. Die Downtime-Freiheit im Sinne dieser Arbeit bewegt sich innerhalb definierter SLO Grenzen, da ein System mit 100% Verfügbarkeit selbst für ein Unternehmen wie Google nicht erreichbar ist. (Vgl. *Key Insights* [12])

Kapitel 3

Konzeption und Methodik

Als sinnbildliches Szenario wurde ein Online-Shop gewählt. Die Architektur des Systems besteht aus zwei Microservices, jeweils ein Repräsentant für den Shop und für die Transaktionen. Zwischen den Services besteht technisch keine Abhängigkeit. Im Microservice für die Transaktionen wird ein simulierter Fehler injiziert, während der Shop-Microservice als Referenz dient, um gegebenenfalls infrastrukturelle Fehler erkennbar zu machen, welche nicht auf die injizierten Fehler zurückzuführen sind. Da es sich hierbei um eine nutzerzentrierte Anwendung handelt, werden zwei Fehlerszenarien überprüft. Einerseits die Injektion einer Fehlerrate, in dem der Service einen internen Serverfehler produziert. Andererseits die Simulation verlängerter Antwortzeit. Es wurde jeweils überprüft, wie lange die jeweilige Analyse benötigt, den Fehler zu erkennen und anschließend den stabilen Zustand widerherzustellen. Weiter wurde berechnet, wie hoch der Verbrauch des Fehlerbudgets innerhalb des getesteten Zeitfensters war.

3.1 Systemarchitektur

Das System wurde in einem AWS EKS Cluster mit zwei m7i-flex.large Nodes aufgesetzt. Diese wurden jeweils dediziert für die Ausführung der Monitoring-Software und Lasttests und die Ausführung der zu testenden Services konfiguriert. Dies verhindert, dass die Messungen durch Konkurrenz um Hardware-Ressourcen beeinflusst werden. Abbildung 3.1 veranschaulicht diesen Zustand noch einmal tiefergehend. Hinzuzufügen ist bei der Systemarchitektur, dass Cilium als Service-Mesh konfiguriert wurde. Dies hätte das Monitoring für die Nutzung der Services

von außerhalb des Clusters vereinfacht. Da die Lasttests jedoch über den Cluster-internen DNS aufgelöst wurden, spielt die Implementierung dessen hier keine weitere Rolle. Die zu testende Applikation besteht aus zwei JVM-basierten Mi-

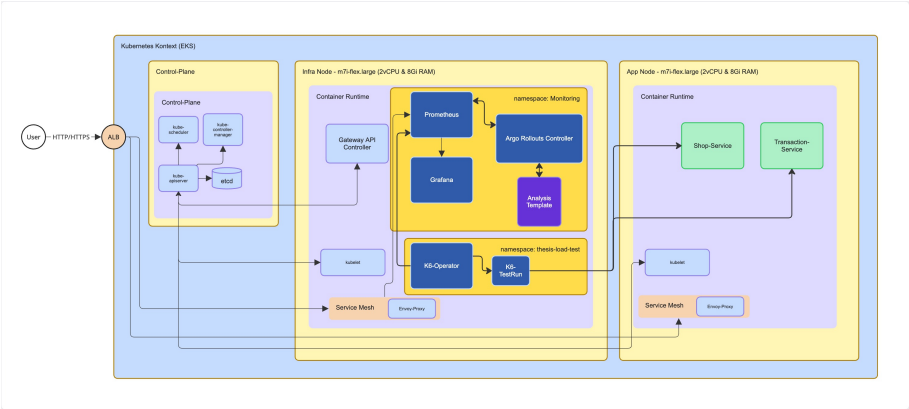

Abbildung 3.1: Systemarchitektur: EKS Cluster Versuchsaufbau

croservices, welche sinnbildlich einen Online-Shop abbilden. Hierfür wurde als Server-Framework Ktor verwendet. Um Fehler während des Deployment Prozesses zu vermeiden, wurden die Container der Microservices jeweils mit Readiness-Probes und einer Mindest-Ready-Zeit von 20 Sekunden ausgestattet. Dies verhindert, dass die Services Anfragen an Pods weiterleten, die sich im Warmup-Prozess befinden. Weiter wurden die Container mit Pre-Stop Hooks von 20 Sekunden versehen, so wird verhindert, dass Anfragen an Pods geleitet werden, die bereits herunterfahren. Um die Erreichbarkeit der Pods sicherzustellen wurden Liveness-Probes implementiert. Diese Applikationen werden auf der Node mit dem Label *role: app-node* gehostet. Die folgenden Software-Tools wurden so konfiguriert, dass sie auf der Node mit dem Label *role: infra-node* ausgeführt werden. Um das Monitoring zu ermöglichen, wurden Prometheus und Grafana im Cluster deployed. Für die Implementierung der jeweiligen Deployment Strategie wurde Argo-Rollouts verwendet. Dies ermöglicht in Verbindung mit Prometheus die SLO-gesteuerte Analyse während des Deployment Vorgangs. Um die Nutzerlast auf den jeweiligen Service zu simulieren wurde K6 mit Constant-Arrival-Rate verwendet. Dies garantiert, dass die Services eine konstant gleichbleibende Last, unabhängig von der Antwortzeit bearbeiten müssen.

3.2 Theoretisches Modell

Rein mathematisch lässt sich bereits beweisen, dass Canary-Releases einen Vorteil beim Verbrauch des Fehlerbudgets gegenüber Rolling Updates haben, in dem Sinne, dass bei gleicher Art des Fehlers, Nutzer kontinuierlich gesteigerten Zugriff auf die neue Softwareversion erhalten. Genauer erklärt am Beispiel: Es wird ein Fehler in der Softwareversion ausgerollt, der in zwei Prozent aller Fälle einen Serverfehler verursacht. Für Rolling Updates erhalten während und nach der Bereitstellung alle Nutzer Zugang zu der neuen fehlerhaften Version, der Einfluss auf den Verbrauch des Fehlerbudgets spiegelt sich direkt wider. Bei Canary Updates werden während des Bereitstellungsprozesses kontinuierlich mehr Nutzer auf die fehlerhafte Version Zugriff erhalten, demnach spiegelt sich während des Deployment Prozesses der Einfluss auf den Verbrauch des Fehlerbudgets anteilig wider. Erst mit dem Abschluss des Canary Releases spiegelt sich der Einfluss direkt auf das Fehlerbudget wider. Weiter ist logisch herzuleiten, dass die Detektionszeit für den Fehler sinkt, wenn sich die Analyse auf die Canary-Version beschränkt, anstatt den Fehler im gesamten System zu überprüfen. Diese Arbeit analysiert den Faktor, um den die Detektionszeit und der Verbrauch des Fehlerbudgets, bei gleicher zeitlicher Analyseintervallrate zwischen Rolling Update und Canary Release mit globaler und isolierter Aggregation sinkt.

3.3 Parameter zur Analyse

Für die Erhebung der Daten wurden jeweils fünf Testläufe über 30 Minuten absolviert. Jeder Service war dabei einer Last von 1200 Anfragen pro Sekunde ausgesetzt. Die injizierten Fehler wurden mittels Umgebungsvariablen realisiert. Da Rolling Updates nativ keine Möglichkeit der Analyse bieten, habe ich mich dazu entschieden, das Alerting von Grafana zu verwenden.
Für die Gewichtung des Datenverkehrs wurden folgende Schritte definiert, nach dem Erhöhen wurde das System für jeweils 5 Minuten überwacht, bevor der nächste Schritt erfolgte:

- 10%
- 50%

- 75%
- 100%

3.3.1 Fehlerfreier Zustand des Systems

Der Zustand der Applikation wurde zuerst ohne injizierte Fehler analysiert, basierend darauf wurde die Definition der SLO's erstellt. Die für diese Arbeit wichtigen Definitionen umfassen:

- eine Verfügbarkeit von 99%
- eine Antwortzeit von 15ms für 95% aller erfolgreichen Anfragen

und beschreiben einen rollierenden Zeitraum von 30 Minuten, im Sinne der Testdauer.

3.3.2 Fehlerszenario A

In Fehlerszenario A wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage in 5% der Fälle einen internen Serverfehler meldete. Die zu überprüfende Metrik im Hinblick auf das SLO, ist das Verhältnis fehlerhafter beantworteter Anfragen zur Summe aller Anfragen.

3.3.3 Fehlerszenario B

In Fehlerszenario B wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage zusätzlich zur normalen Antwortzeit 15ms warten musste. Die zu überprüfende Metrik ist die Antwortzeit über 95% aller Anfragen, die ohne Fehler beantwortet wurden.

3.3.4 Analyseintervalle

Um ein einheitliches Analyse-Verfahren zu garantieren, wurden die Werte für die Analyseintervalle für alle Analysemethoden vereinheitlicht. Alle 60 Sekunden findet die Überprüfung der Metrik statt. Während für die Analysistemplates ein Feh-

lerlimit von eins gesetzt wurde, was eine einmalige Überschreitung des Schwellwerts zulässt, wurde der Alertmanager genauso konfiguriert, sodass er bei der ersten Schwellwertüberschreitung den Pending Status annimmt, demnach zulässt und erst bei der zweiten Schwellwertüberschreitung den alarmierten Zustand auslöst.

Kapitel 4

Evaluation

Für die Fehlerszenarien wurden die Zeit bis zur Detektion und die Zeit bis zur Wiederherstellung des stabilen Zustands mithilfe der Kubernetes Events bestimmt. Weiter werden die einzelnen Testläufe im zeitlichen Verlauf nebeneinander dargestellt und abschließend werden die gemittelten Werte der unterschiedlichen Deployment Strategien im Kombinationsdiagramm miteinander verglichen.

4.1 Analyse der injizierten Fehlerrate

4.1.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit

Abbildung 4.1 zeigt, dass die Zeit bis zur Detektion für Rolling Updates im Vergleich zu den Canary Methoden mit rund 3,3 Minuten am geringsten war. Gefolgt von der Canary Methode mit isolierter Metrikanalyse mit rund 8 Minuten, am längsten benötigte die Canary Methode mit globaler Analysemetrik, hier betrug die Zeit bis zur Detektion im Schnitt 12 Minuten. Die geringste Wiederherstellungszeit erreichte die Canary Methode mit isolierter Metrikanalyse, im Durchschnitt betrug die Zeit rund eine halbe Minute. Die zeitlichen Unterschiede lassen sich mit dem Versuchsaufbau erklären. Bei den Rolling Updates wurde der Fehler sofort erkennbar, da 5% aller Anfragen an das System fehlschlugen. Der zeitliche Unterschied zwischen den Canary Release Methoden lässt sich mit der verwendeten Analysemetrik begründen. Für die globale Aggregation musste die Gewichtung des Datenverkehrs erst einen signifikanten Schwellwert überschreiten, dass die

Fehler nicht von den fehlerfreien Pods überlagert wurden. Die isolierte Analyse stellte hingegen das direkte Verhältnis der fehlerhaften Anfragen zu den Gesamtanfragen an die Canary Version gegenüber und konnte daher einen zeitlichen Vorteil erlangen. Diskutiert wird später dennoch, weshalb während der initialen 10% Gewichtung keine Fehler erkannt wurden, obwohl diese mathematisch Vorkommen sollten. Die Canary Methode mit globaler Analysemetrik stellte den stabilen Zustand innerhalb von rund einer Minute wieder her. Das Rolling Update benötigte für die Wiederherstellung des stabilen Zustands fast zwei Minuten. Die Wiederherstellungszeiten lassen sich in Zusammenhang mit der Anzahl an Pods bringen, die ausgetauscht werden mussten, um die fehlerfreie Version wieder lauffähig zu machen. Die Rolling Updates mussten der Reihe nach alle Pods ersetzen, während bei den Canary Release Methoden noch fehlerfreie Pods lauffähig waren. Der Zeitunterschied von isolierter und globaler Analysemetrik lässt sich weiter in Zusammenhang mit den fehlerfrei laufenden Pods bringen. Die Canary Methode mit isolierter Metrikanalyse detektierte die Fehler im zweiten Schritt, hier waren 50% User-Traffic auf die neue Version umgeleitet und die fehlerfreie Pod Anzahl betrug zwei. Die Canary Methode mit globaler Analysemetrik detektierte den Fehler erst im dritten Schritt mit 75% User-Traffic, die fehlerfreie Pod Anzahl betrug eins.

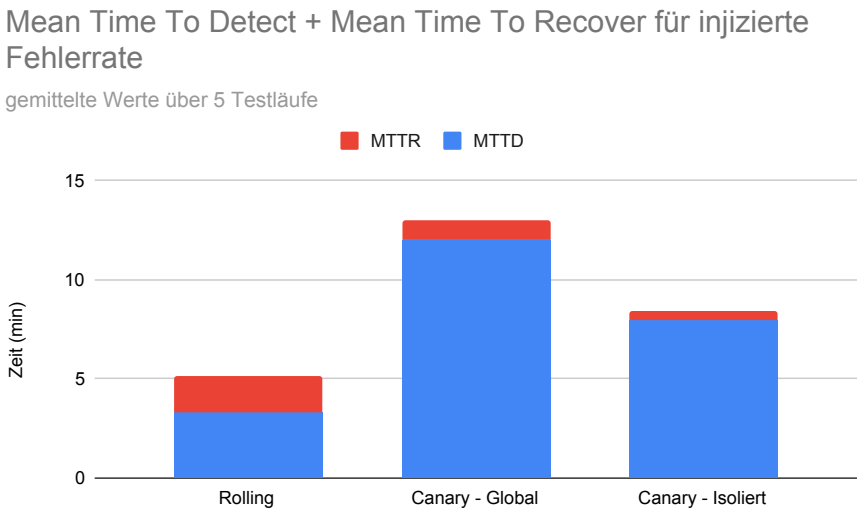

Abbildung 4.1: Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Fehlerrate

4.1.2 Die einzelnen Testläufe im zeitlichen Verlauf

Die Diagramme zeigen den globalen Einfluss der injizierten Fehlerrate auf das System. Beginnend mit dem Vergleich der einzelnen Testläufe für das Rolling Update zeigt Abbildung 4.2, dass die Fehlerrate den Maximalwert von 5% innerhalb von 5 Minuten erreichte und dieser nach Abschluss des Rollbacks wieder auf null sank.

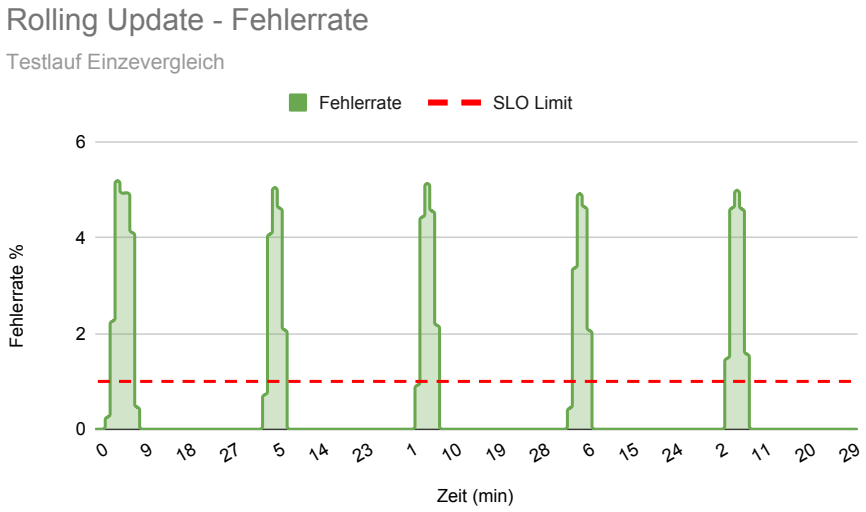

Abbildung 4.2: Fehlerrate Rolling Update im Einzelvergleich

In Abbildung 4.3 wird ersichtlich, dass die Canary Release Strategie mit globaler Metrikanalyse einen Maximalwert von 2,3% Fehlerrate innerhalb von 13 Minuten erreichte. Auffällig ist der erste Testlauf, der nach elf Minuten seinen Peak von 1,1% Fehlerrate erreichte und im Gegensatz zu den folgenden Testläufen direkt zurückgerollt wurde. Inwiefern die Analyse hier schneller reagiert, wird später diskutiert.

Während beide vorherigen Deployment Strategien das SLO-Limit überschritten, zeigt Abbildung 4.4, dass die isolierte Analysemetrik das SLO-Limit erfolgreich über alle Testläufe eingehalten hat. Hier wurden die Maximalwerte im Schnitt innerhalb von neun Minuten erreicht.

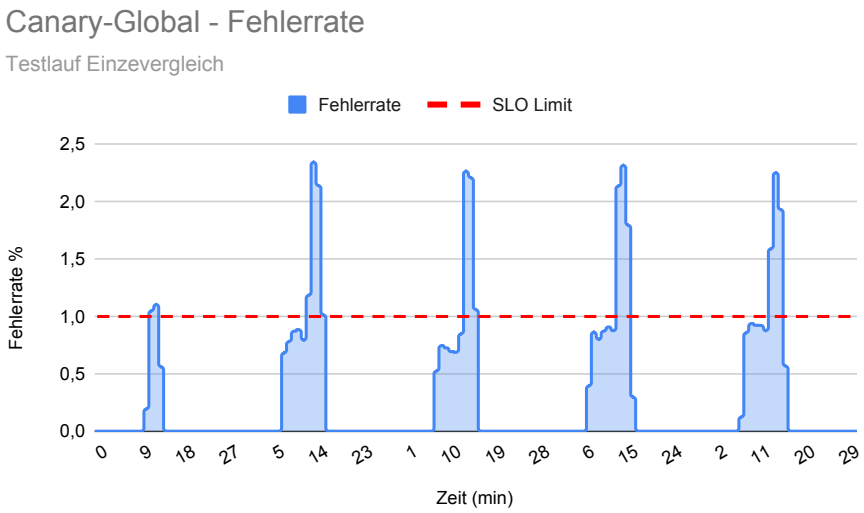

Abbildung 4.3: Fehlerrate Canary Release globaler Metrikanalyse im Einzelvergleich

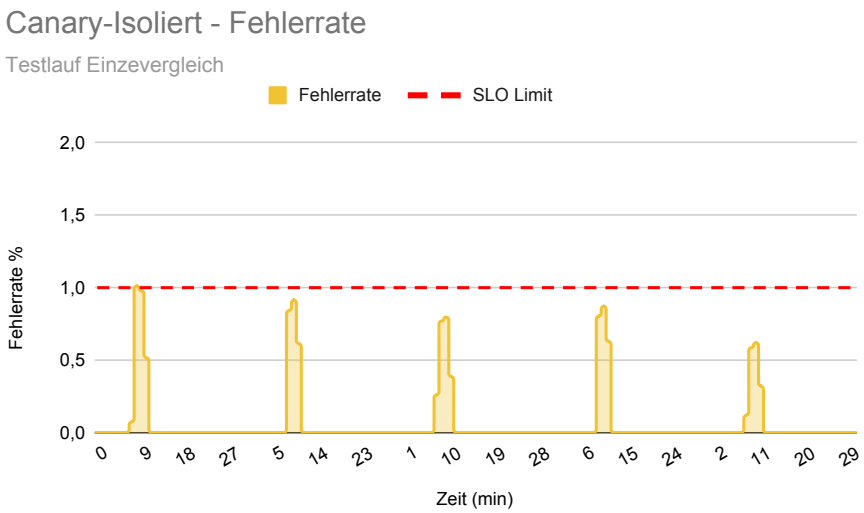

Abbildung 4.4: Fehlerrate Canary Release isolierter Analysemetrik im Einzelvergleich

4.1.3 Die gemittelten Werte der Deployment Strategien im Vergleich

Deutlicher werden die ermittelten Werte im direkten Vergleich der Deployment Strategien. In Abbildung 4.5 wird der Zusammenhang aus Zeit bis zur Detektion und Einfluss der injizierten Fehlerrate für die unterschiedlichen Deployment Strategien stärker ersichtlich, als bei der Einzelbetrachtung. Während das Rolling Update bereits den Wiederherstellungsprozess der stabilen Version abgeschlossen hatte, entstanden die Fehler der Canary Methoden erst. Dafür begrenzte sich der Einfluss der injizierten Fehlerrate bei den Canary Methoden, während das Rolling Update die vollen 5% injizierte Fehlerrate aufwies.

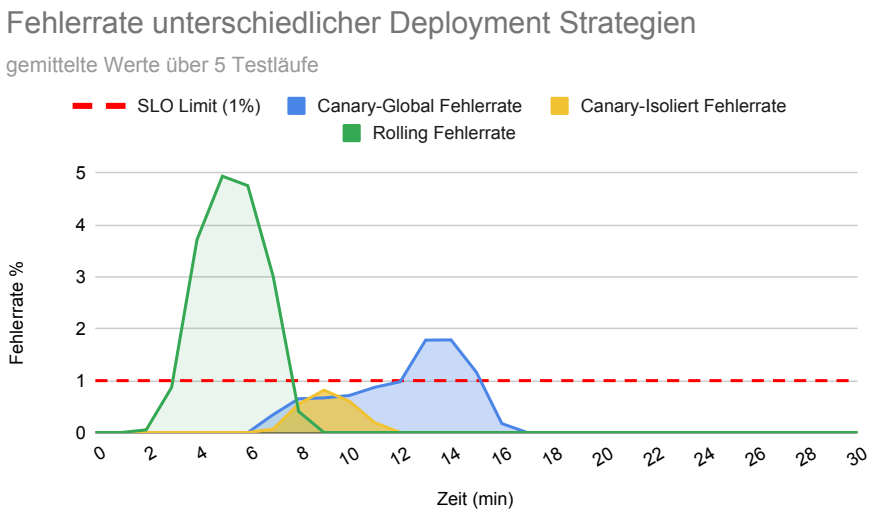

Abbildung 4.5: Fehlerrate unterschiedlicher Deployment Strategien

4.2 Analyse der injizierten Latenz

4.2.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit

Abbildung 4.6 zeigt auch hier, dass das Rolling Update die geringste Zeit von der Fehlerinjektion bis zur Wiederherstellung des stabilen Zustands benötigte. Im Durchschnitt über alle Testläufe rund 6,9 Minuten. Das Canary Release mit isolierter Analysemetrik benötigte von der Injektion bis zur Wiederherstellung rund

7,3 Minuten, während das Canary Release mit globaler Metrikanalyse mit rund 13,3 Minuten am längsten benötigte. Wie auch im Fehlerszenario A lassen sich diese Zeiten mit dem direkten Einfluss des injizierten Fehlers für das Rolling Update erklären. Gleich gilt auch der Vorteil der isolierten Analyse der Canary Releases gegenüber dem globalen Ansatz, da für die isolierte Metrik, der direkte Einfluss der Canary Version überprüft wurde, hingegen musste bei der globalen Metrikanalyse erst eine ausreichend hohe Gewichtung des Datenverkehrs auf die Canary Version erreicht werden, damit der Fehler erkannt wurde.

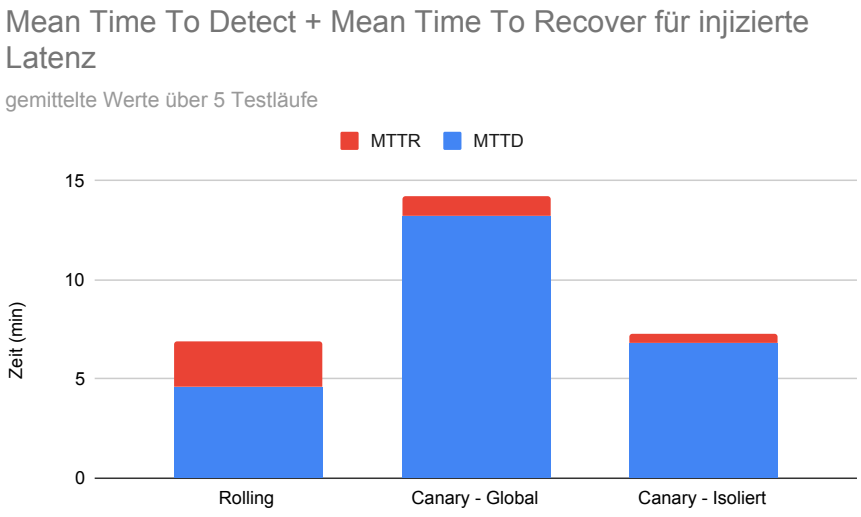

Abbildung 4.6: Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Latenz

4.2.2 Die einzelnen Testläufe im zeitlichen Verlauf

Die Diagramme zeigen den globalen Einfluss der injizierten Fehlerrate auf das System. Beginnend mit dem Vergleich der einzelnen Testläufe für das Rolling Update zeigt Abbildung 4.7, dass die Latenz Maximalwerte von über einer Sekunde Antwortzeit betrug. Dies ist mit blockierenden Threads zu erklären. Die Softwareversion hat es nicht geschafft die Anfragen in genügend geringer Zeit zu beantworten und die Pods wurden durch die Überlastung von CPU-Ressourcen kontinuierlich neu gestartet. Diese Unerreichbarkeit der Services ist als Downtime zu bewerten.

In Abbildung 4.8 wird ersichtlich, dass die Canary Release Strategie mit globaler Metrikanalyse einen Maximalwert von bis zu 290 Millisekunden an Antwortzeit

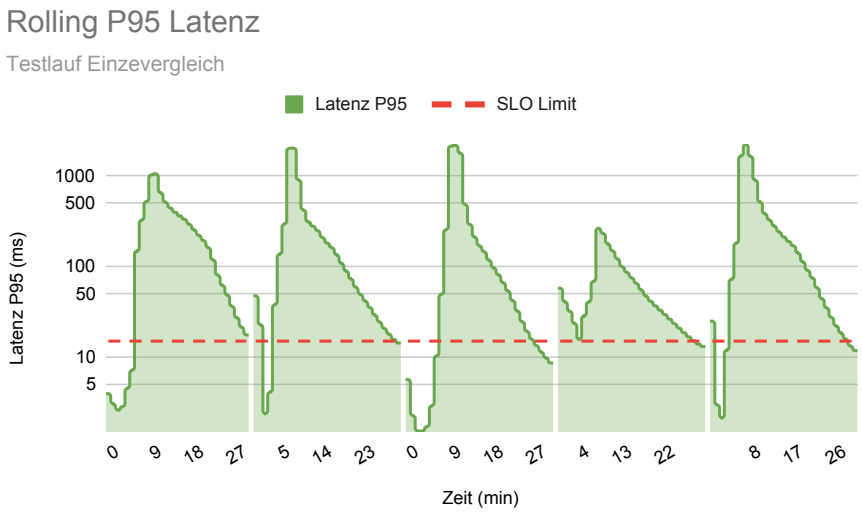

Abbildung 4.7: Latenz Rolling Update im Einzelvergleich

benötigte. Dies entspricht einer Steigerung um den Faktor 20 der erlaubten Antwortzeit entsprechend des SLO.

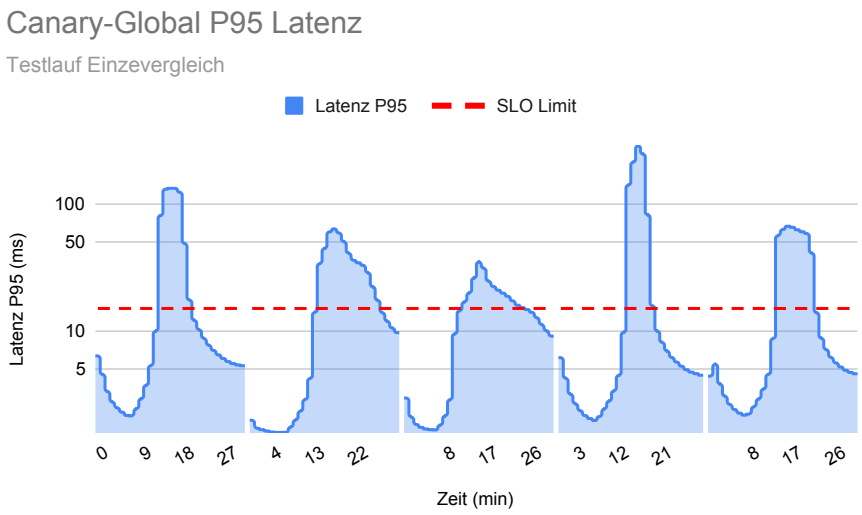

Abbildung 4.8: Latenz Canary Release globaler Metrikanalyse im Einzelvergleich

Die beiden vorherigen Deployment Strategien überschritten das SLO-Limit deutlich, Abbildung 4.9 zeigt, dass die isolierte Analysemetrik das SLO-Limit erfolgreich über alle Testläufe eingehalten hat. Auffällig sind im Diagramm einzig die Warm-Up Phasen während des Deploymentstarts.

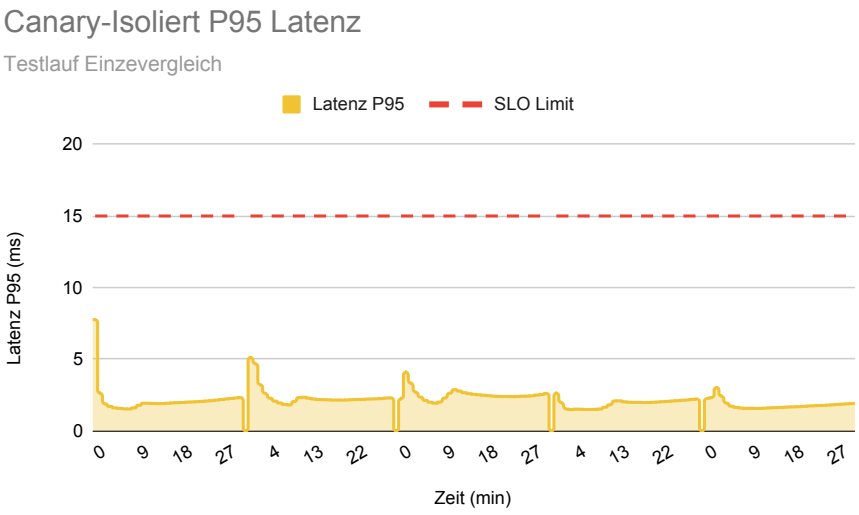

Abbildung 4.9: Latenz Canary Release isolierter Analysemetrik im Einzelvergleich

4.2.3 Die gemittelten Werte der Deployment Strategien im Vergleich

Der direkte Vergleich der Deployment Strategien bringt die Überlegenheit des isolierten gegenüber des globalen Analyseansatzes und der Analyse des Rolling Updates deutlich zum Vorschein. Die globale Analyse des Canary Releases und das Rolling Update überschritten das gesetzte Limit bei Weitem. Während die isolierte Metrikanalyse des Canary Releases keinen Einfluss auf das Gesamtsystem aufweist.

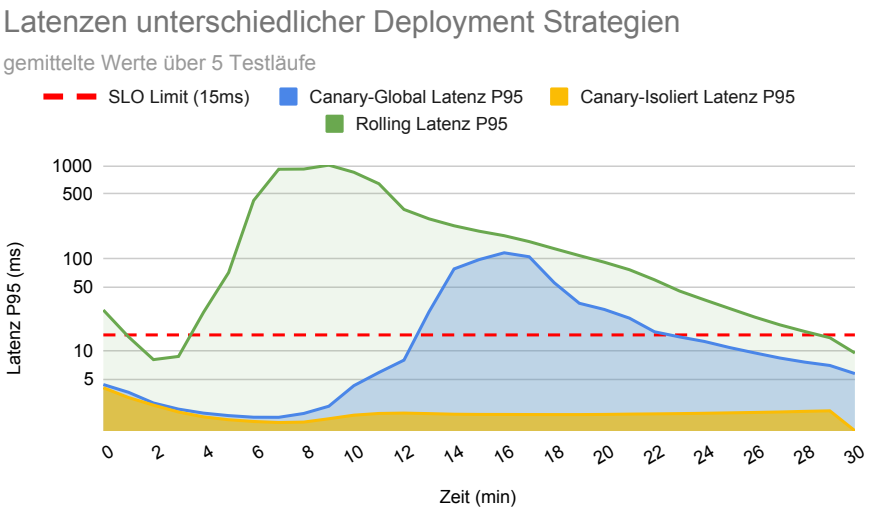

Abbildung 4.10: Latenzen unterschiedlicher Deployment Strategien

4.3 Fehlerbudget Verbrauch pro Deployment Strategie

4.3.1 Fehlerszenario A

Abbildung 4.11 zeigt den Einfluss der Analysemetrik und der Deployment Strategie auf den Verbrauch des Fehlerbudgets. Den geringsten Verbrauch erreichte die Canary Release Strategie mit isolierter Analyse. Der Bedarf betrug 7,4% des Gesamtbudgets. Das Canary Release mit globaler Analyse erreichte einen Bedarf von 30,42% am Gesamtbudget. Den höchsten Verbrauch des Fehlerbudgets bedurfte das Rolling Update mit 59,14%. In diesem speziellen Szenario verringerte die Canary Release Strategie mit globaler Analysemetrik den Verbrauch des Fehlerbudgets im Gegensatz zum Rolling Update um 28,71 Prozentpunkte und die isolierte Analyse erreichte eine Verringerung des Budgetverbrauchs gegenüber dem Rolling um 51,72 Prozentpunkte. Weiter ist eine Verringerung des isolierten gegenüber dem globalen Ansatz um 23,01 Prozentpunkte festzuhalten.

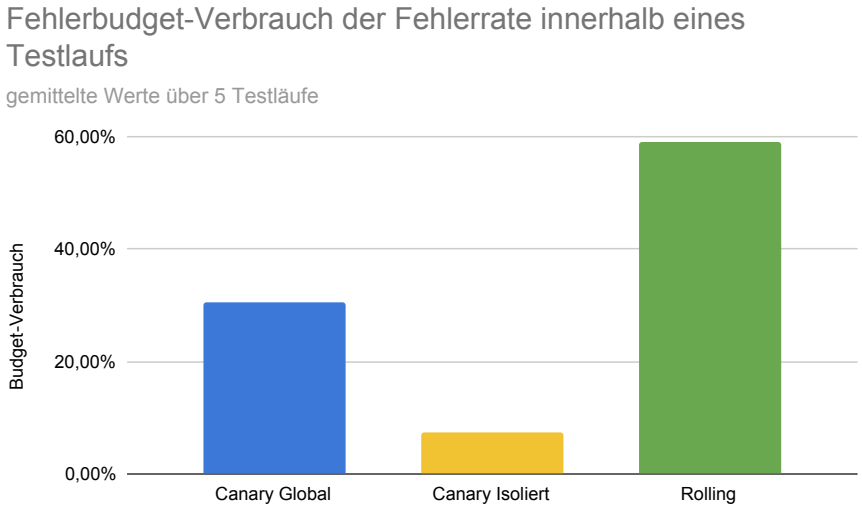

Abbildung 4.11: Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Fehlerrate über die Testdauer

4.3.2 Fehlerszenario B

Abbildung 4.12 zeigt ebenfalls den Einfluss der Analysemetrik und Deployment Strategie hinsichtlich des Verbrauchs des Fehlerbudgets. Während der geringste

Bedarf von 0,25% am Fehlerbudget durch die isolierte Analysemetrik des Canary Releases erreicht wurde, verbrauchte die globale Analyse 2,59%. Das Rolling Update stellte mit rund 26% den höchsten Bedarf am Fehlerbudget. Die Einsparung des Fehlerbudgets von Canary Release mit globaler Analyse beläuft sich auf 23,4 Prozentpunkte gegenüber dem Rolling Update. Das Canary Release mit isolierter Analyse erreichte eine Einsparung von 25,74 Prozentpunkten gegenüber dem Rolling Update. Die Verringerung der Canary Release Strategie mit isolierter gegenüber globaler Analyse beläuft sich auf 2,34 Prozentpunkte.

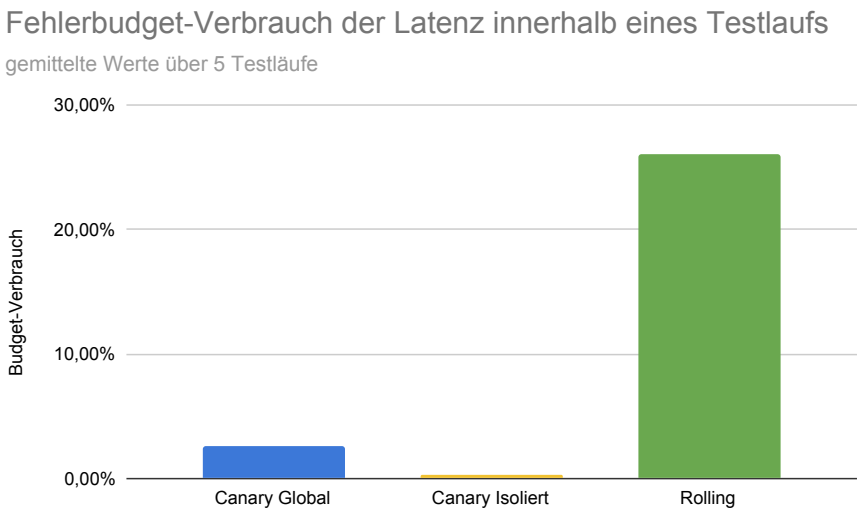

Abbildung 4.12: Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Latenz über die Testdauer

Kapitel 5

Diskussion

5.1 Der erste Testlauf der injizierten Fehlerrate für das Canary Release mit globaler Metrikanalyse

Beim Erstellen der Graphen für die einzelnen Testläufe war dieser direkt auffällig. Offensichtlicher wurde die Auffälligkeit nach der Erstellung der Analyse der Canary Releases mit isolierter Metrik. Die Werte stimmten beinahe überein. Von der Erklärung abgesehen, dass die Messwerte hier zufällig innerhalb von zwei Minuten den Schwellwert geringfügig überschritten, vermute ich, dass die Beachtung der JVM-Warmup-Zeiten eine Verbesserung der Detektionszeit hervorgerufen hätte, in dem Sinne, dass die Analyse mit einer initialen Verzögerung Fehler schneller detektiert hätte, da die ersten Pods bereits lauffähig und ansprechbar gewesen wären.

5.2 Nicht gemessene Fehler während der ersten Canary Phase

Bei der Analyse der Daten ist aufgefallen, dass praktisch keine Fehlerrate während der initialen Canary Phase, der Gewichtung des Datenverkehrs von 10%, erfasst wurde. Mathematisch betrachtet, sollten mit 1200 Anfragen pro Sekunde, einer Gewichtung von 10% und einer Fehlerrate von 5%, sechs Fehler pro Sekunde und dementsprechend 360 Fehler pro Minute auftreten, jedoch erfasste

weder die globale, noch die isolierte Analyse einen einzigen Fehler für die Gewichtung während der Zeit von fünf Minuten. Ein möglicher Grund für das Nicht-Erfassen der globalen Analyse dieser Fehlerrate kann statistische Irrelevanz sein. 360 Fehler zu 72.000 Anfragen pro Minute ist für die Aggregation der Metrik möglicherweise nicht signifikant genug. Andernfalls erklärt das die Nicht-Erkennung der isolierten Analyse nicht, da hier 360 Fehler zu 7.200 Anfragen stehen, was genau der injizierten Fehlerrate entspricht. Eine genaue Erklärung der Nicht-Erkennung lässt sich aus meiner Sicht nicht treffen.

5.3 Detektionszeit in Zusammenhang des Fehlerbudgetverbrauchs

Die Berechnung der Zeit bis zur Detektion des Fehlers zeigt deutlich, dass die Rolling Updates Analyse den Fehler am schnellsten erkannt hat. Jedoch hat diese Deployment Strategie auch den größten Verbrauch am Fehlerbudget. Wie im oberen Abschnitt beschrieben, wurden für die initiale Phase des Canary Releases jedoch kaum bis keine Defekte der Softwareversion festgestellt. Würde die Zeit der initialen Phase von der Detektionszeit abgezogen werden, so hätte die Canary Release Strategie mit isolierter Analyse eine geringere Detektionszeit gegenüber dem Rolling Update. Andersherum hätte die Datenverkehr Gewichtung innerhalb der initialen Phase auch schneller erhöht werden können, um einen statistisch signifikanten Schwellwert zu überschreiten und schneller Fehler zu provozieren. Letztlich zeigen die Daten der Detektionszeit im Zusammenhang mit dem Fehlerverbrauch, dass eine schnellere Detektionszeit nicht direkt mit einer Ersparnis des Fehlerbudgets einhergehen, sondern dass der Anteil betroffener Nutzer den entscheidenden Anteil am Verbrauch des Fehlerbudgets trägt.

Kapitel 6

Fazit

Abschließend werden die Forschungsfrage beantwortet und ein Ausblick über mögliche sich dieser Arbeit anschließende Forschungsprojekte benannt.

6.1 Beantwortung der Forschungsfrage

Im speziellen Versuchsaufbau von 1200 Anfragen pro Sekunde auf den jeweiligen Service und injizierten Fehlern von fünf Prozent Fehlerrate und 15 Millisekunden zusätzlicher Latenz, konnte das Canary Release mit SLO basierter Analyse bei gleicher überprüfter Metrik eine Reduzierung des Fehlerbudgetverbrauchs gegenüber Rolling Updates von 28% für die injizierte Fehlerrate und 23% für injizierte Latenz erreichen. Durch den Einsatz isolierter Analysen für Canary Releases war es möglich das Fehlerbudget um 51% bei injizierter Fehlerrate und 25% bei injizierter Latenz zu verringern.

6.2 Ausblick

In weiteren analytischen Arbeiten kann überprüft werden, inwiefern sich die Detektionszeit bei feinerer Gewichtung des Datenverkehrs verringern lässt, welchen Einfluss das Analyseintervall auf die Detektionszeiten hat oder inwiefern sich Deployments mit Blue-Green Strategie unterscheiden. Weiter könnte der Kostenfaktor analysiert werden, der durch den Mehraufwand dedizierter Monitoring Nodes entsteht und anschließend in Verhältnis zur Ersparnis gesetzt werden.

Literatur

[1] Argo Rollouts Autoren. *Concepts*. 2016. URL: https://argo-rollouts.readthedocs.io/en/stable/concepts/#deployment-strategies (besucht am 20. 03. 2026).

[2] Argo Rollouts Autoren. *Concepts*. 2016. URL: https://argo-rollouts.readthedocs.io/en/stable/concepts/#canary (besucht am 20. 03. 2026).

[3] AWS Autoren. *What is Site Reliability Engineering (SRE)?* 2026. URL: https://aws.amazon.com/what-is/sre/ (besucht am 21. 03. 2026).

[4] Chris Jones, John Wilkes, Niall Murphy, Cody Smith, Betsy Beyer. *Service Level Objectives*. 2017. URL: https://sre.google/sre-book/service-level-objectives/ (besucht am 21. 03. 2026).

[5] Danilo Sato. *Canary Release*. 2014. URL: https://martinfowler.com/bliki/CanaryRelease.html (besucht am 06. 02. 2026).

[6] Google SRE Autoren. *Part II. Principles*. 2017. URL: https://sre.google/sre-book/part-II-principles/ (besucht am 21. 03. 2026).

[7] James Lewis, Martin Fowler. *Microservices*. 2014. URL: https://www.martinfowler.com/articles/microservices.html (besucht am 17. 01. 2026).

[8] Kubernetes Autoren. *Kubernetes*. 2026. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment (besucht am 22. 02. 2026).

[9] Kubernetes Autoren. *Overview*. 2026-03-01. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 21. 03. 2026).

[10] Kubernetes Autoren. *Service*. 2026-01-07. URL: https://kubernetes.io/docs/concepts/services-networking/service/ (besucht am 21. 03. 2026).

[11] Kubernetes Autoren. *Workloads*. 2025-11-18. URL: https://kubernetes.io/docs/concepts/workloads/ (besucht am 21. 03. 2026).

[12] Marc Alvidrez, Kavita Guliani, Mark Roth, Carmela Quinito. *Embracing Risk*. 2017. URL: https://sre.google/sre-book/embracing-risk/ (besucht am 21. 03. 2026).

[13] RedHat Autoren. *Was ist SRE (Site Reliability Engineering)?* 2023-03-24. URL: https://www.redhat.com/de/topics/devops/what-is-sre (besucht am 21. 03. 2026).

[14] Betsy Beyer Rob Ewaschuk. *Monitoring Distributed Systems*. 2017. URL: https://sre.google/sre-book/monitoring-distributed-systems/ (besucht am 21. 03. 2026).

[15] Wikipedia. *Downtime*. 2022-08-31. URL: https://de.wikipedia.org/wiki/Downtime (besucht am 21. 03. 2026).

Anhang A

Eigenständigkeitserklärung

Hiermit erkläre Ich, dass die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Stellen, die wörtlich oder sinngemäß aus Quellen entnommen wurden, sind als solche kenntlich gemacht. Diese Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Borna, den 24.03.2026 Paul Fuchs